

TinyML Badminton Coach

*Real-time Badminton Smash Classification Using IMU Data and
On-Device TinyML Inference*

Team Members:

Momoka Sakamoto

Bobby Taing

Rania Sundramurti

Course: EE 446: TinyML for Ultra Edge Computing

Quarter: Spring 2026

*Department: Electrical and Computer Engineering, University of
Washington*

Team Contributions:

Bobby: Contributed to the TinyML model comparison, INT8 quantization analysis, and final deployment workflow. Evaluated the spectral-feature pipeline against raw-data pipeline, compared Float32 and INT8 model results, and tested the deployed Arduino model using live Serial Monitor outputs.

Momoka: Contributed TinyML model development and evaluation process. Conducted comparison between spectral-features and raw-data classifier. Contributed to system planning, configured the Edge Impulse training pipeline, and prepared the dataset for Edge Impulse compatibility.

Rania: Led dataset collection and preprocessing efforts. Developed and tested the IMU data acquisition workflow on Arduino Nano 33 BLE Sense Rev2, collected and organized badminton smash datasets across the all the class. Contributed to dataset preparation and experimental setup for model training.

Introduction and Project Objective:

Badminton smash quality can be difficult for players to evaluate during practice without a coach, video review, or external feedback system. A player, especially beginners, may not immediately know whether a swing was a strong smash, a weak smash, or not a valid smash. This creates an opportunity for a portable embedded system that can classify swing motion and provide fast feedback during training.

The objective of this project is to build a TinyML-based badminton smash classifier using IMU data collected from the Arduino Nano 33 BLE Sense Rev2. The input to the system is a 1-second window of accelerometer and gyroscope data from the onboard IMU. The output is a three-class prediction: *good_smash*, *weak_smash*, or *non_smash*.

The task is suitable for TinyML because badminton swing classification can be performed using short motion windows and does not require cloud processing. Running inference directly on the microcontroller makes the system portable, low-latency, and usable during practice without a laptop or external server. The target deployment platform is the Nano 33, with inference results shown through the Arduino Serial Monitor.

The success criteria for this project are based on both model performance and embedded deployment. The model should achieve strong classification accuracy and F1 score on the test set, remain small enough to fit on the target hardware, and run with low enough latency for real-time feedback. The final system should also demonstrate successful on-device inference using live or realistic IMU input.

Dataset and Experimental Setup:

The dataset was collected using the onboard IMU of the Arduino Nano 33 BLE Sense Rev2. The IMU provided both accelerometer and gyroscope readings during badminton swing motions. Each recorded sample represented one swing attempt and was labeled as one of three classes: *good_smash*, *weak_smash*, or *non_smash*.

Each sample used six sensor channels: *aX*, *aY*, *aZ*, *gX*, *gY*, and *gZ*. The input window length was 1000 ms with a sampling rate of 100 Hz, so each sample contained 100 time steps. Since there were six sensor channels, each raw sample contained 600 sensor values before feature extraction.

The final dataset contained 60 total samples. The data was split into 47 training samples and 13 testing samples. The split was performed at the sample level, meaning complete swing windows were assigned either to training or testing. This helped avoid data leakage because overlapping portions of the same swing were not shared between the training and test sets.

Preprocessing was handled through Edge Impulse. The uploaded IMU samples were treated as 1-second motion windows using a 1000 ms window size and 1000 ms window increase. Since the window size and window increase were equal, each sample was treated as a separate non-overlapping window. No manual data augmentation was used; instead, the dataset focused on clean and consistently labeled examples for each class.

The evaluation used accuracy, weighted F1 score, and a confusion matrix to measure classification performance across the three swing classes. These metrics were selected because the task is a three-class classification problem, and the confusion matrix helps show which swing types are most often confused.

Model Development and Advanced Components:

Model development was performed using Edge Impulse for feature extraction, neural network training, testing, quantization, and Arduino deployment.

The main baseline model used spectral feature extraction followed by a neural network classifier. Spectral analysis transformed the raw IMU time-series data into frequency-domain features, which helped summarize swing motion patterns such as intensity and motion changes. The spectral classifier used 78 input features, followed by a dense layer with 20 neurons, a dense layer with 10 neurons, and a 3-class output layer. The model was trained for 100 cycles with a learning rate of 0.005 using the Edge Impulse CPU training processor.

A second model pipeline was trained using raw time-series data directly. This raw-data classifier used the same neural network structure, but the input layer contained 600 features because it used the full 1-second IMU window directly. This model was included as an alternative approach

to evaluate whether raw sensor values or extracted spectral features were better for badminton smash classification.

The first advanced component was the comparison between spectral features and raw-data classification. The spectral model achieved 90.0% accuracy and 0.90 F1 score, while the raw-data model achieved 80.0% accuracy and 0.80 F1 score. The spectral model also used less memory, with 1.4 KB RAM and 15.8 KB Flash compared to 2.0 KB RAM and 26.0 KB Flash for the raw-data model. Based on this comparison, the spectral feature pipeline was selected as the stronger model approach.

Model / Pipeline	Test Accuracy	F1 Score	RAM	Flash / Model Size	Latency	Deployed?
Spectral Analysis + Classifier	90.0%	0.90	1.4 KB	15.8 KB	1 ms	Yes
Raw Data + Classifier	80.0%	0.80	2.0 KB	26.0 KB	1 ms	No
INT8 Quantized Model	92.31%	0.92	3.2 KB	15.8 KB	35 ms total	Yes

Figure 1: Spectral Features vs Raw-Data Model Classifications

The second advanced component was model compression using INT8 quantization. After selecting the spectral pipeline, the Float32 model was compared against the INT8 quantized model. The Float32 model achieved 92.31% accuracy with 1.6 KB classifier RAM and 17.7 KB Flash. The INT8 model kept the same 92.31% accuracy while reducing classifier RAM to 1.4 KB and Flash to 15.8 KB. The total latency remained 35 ms for both versions.

Model	Accuracy	RAM	Flash	Latency
Float32	92.31%	1.6 KB	17.7 KB	35 ms
INT8	92.31%	1.4 KB	15.8 KB	35 ms

Figure 2: Float32 vs INT8 Comparison

The final deployed model was selected as the spectral-analysis INT8 quantized classifier. This was chosen because it had the best balance between accuracy, F1 score, memory usage, and deployment efficiency. Spectral features improved classification performance compared to raw data, while INT8 quantization reduced memory usage without reducing accuracy. This made the final model suitable for deployment on the Nano 33.

TinyML System Design and On-Device Deployment:

The end-to-end pipeline begins with IMU sensing, where the onboard accelerometer and gyroscope collect motion data during a badminton swing. The sensor readings are arranged into a 1-second window using six input channels: *aX*, *aY*, *aZ*, *gX*, *gY*, and *gZ*. This input window is then passed through the Edge Impulse spectral analysis block, followed by the trained neural network classifier. The final output is a class prediction for *good_smash*, *weak_smash*, or *non_smash*, which is printed through the Arduino Serial Monitor.

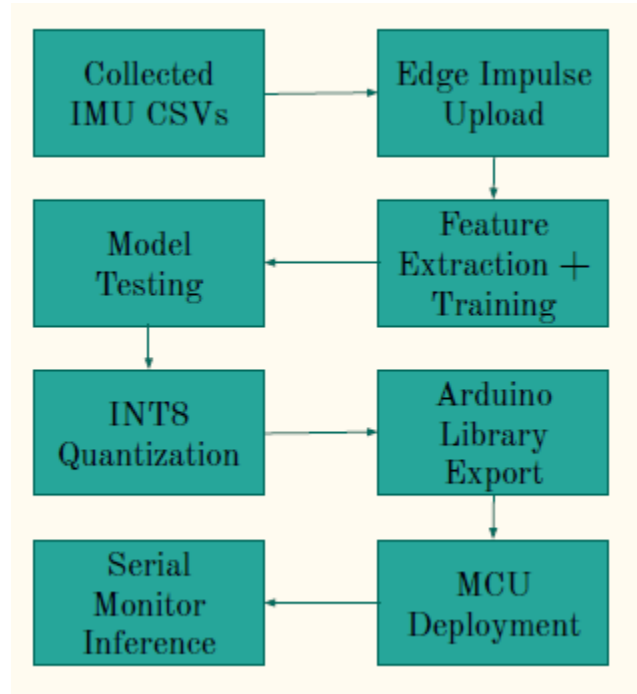


Figure 3: System Pipeline

After training and testing the model in Edge Impulse, the selected spectral classifier was converted into a TinyML-ready Arduino library. The final deployed version used the INT8 quantized model because it preserved the same test accuracy as the Float32 model while reducing classifier RAM and Flash usage. Edge Impulse generated the Arduino deployment files, including the model data, inference code, and library header used in the Arduino sketch. The sketch included the Edge Impulse inference library and ran inference locally on the Arduino board.

During deployment, the Arduino sensor fusion example was used and adjusted so that the sensor names matched the dataset headers used during training (i.e. *accX* -> *aX* & *gyrX* -> *gX*). This allowed the live IMU readings to match the model input format.

The deployed model used the Arduino Nano 33 BLE Sense Rev2 as the target hardware. Edge Impulse estimated the final INT8 model with 35 ms total latency, including 34 ms for spectral feature extraction and 1 ms for classification. The final deployment used 3.2 KB total RAM and

15.8 KB Flash for the classifier. The exported model metadata reported a TFLite arena size of 3020 bytes, which is consistent with the 3.2 KB total RAM estimate. These results show that the model was small enough and fast enough for on-device inference on the target microcontroller.

On-device inference was verified using the Arduino Serial Monitor. During live testing, the board collected IMU input and printed class probabilities for *good_smash*, *weak_smash*, and *non_smash*. The Serial Monitor outputs showed that the deployed model could produce real-time predictions from physical motion input. This confirmed that the project completed the full TinyML pipeline from data collection and model training to quantization, Arduino deployment, and live embedded inference.

```
Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 87 ms., Classification: 1 ms., Anomaly: 0 ms.):
good_smash: 0.99609
non_smash: 0.00000
weak_smash: 0.00000

Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 87 ms., Classification: 1 ms., Anomaly: 0 ms.):
good_smash: 0.99609
non_smash: 0.00000
weak_smash: 0.00000

Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 87 ms., Classification: 1 ms., Anomaly: 0 ms.):
good_smash: 0.99609
non_smash: 0.00000
weak_smash: 0.00000
```

Figure 4: Live Arduino Serial Monitor output for *good_smash*

```
Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 87 ms., Classification: 1 ms., Anomaly: 0 ms.):
good_smash: 0.00781
non_smash: 0.04688
weak_smash: 0.94531

Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 87 ms., Classification: 1 ms., Anomaly: 0 ms.):
good_smash: 0.03516
non_smash: 0.01172
weak_smash: 0.95312

Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 87 ms., Classification: 1 ms., Anomaly: 0 ms.):
good_smash: 0.04688
non_smash: 0.00391
weak_smash: 0.94922
```

Figure 5: Live Arduino Serial Monitor output for weak_smash

```
Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 87 ms., Classification: 1 ms., Anomaly: 0 ms.):
good_smash: 0.02344
non_smash: 0.91016
weak_smash: 0.06641

Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 87 ms., Classification: 1 ms., Anomaly: 0 ms.):
good_smash: 0.00000
non_smash: 0.69531
weak_smash: 0.30469

Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 87 ms., Classification: 1 ms., Anomaly: 0 ms.):
good_smash: 0.27734
non_smash: 0.71094
weak_smash: 0.01172
```

Figure 6: Live Arduino Serial Monitor output for non_smash

Results and analysis:

The final deployed model achieved strong performance on both the validation and test datasets. The selected model combined spectral feature extraction with an INT8-quantized neural network classifier. During validation, the model achieved 90.0 % accuracy and a weighted F1 score of 0.90. On the test dataset, the quantized model achieved 92.31 % accuracy with a weighted precision of 0.94, weighted recall of 0.92, and weighted F1 score of 0.92.

This result indicates that the model was highly effective at recognizing badminton smash motions from IMU sensor data. The good_smash and non_smash classes achieved the best classification performance on the test set. The weak_smash class was more challenging because its motion patterns partially overlapped with non_smash examples. As a result, a weak_smash sample was classified as non_smash, reducing the weak_smash, which reduced the weak_smash accuracy to 75 %. Despite this error, the overall classification performance remained strong across all three classes.

Comparing the spectral-feature model and the raw-data model, the spectral model achieved 90.0 % accuracy and a 0.90 weighted F1 score, which is much better than raw-data model which achieved 80.0 % accuracy and 0.80 weighted F1 score. The spectral model also required less memory and flash storage and it is more suitable for deployment on a resource-constrained microcontroller.

The INT8 quantization experiment further showed that model compression can reduce resource usage without sacrificing performance. The quantized model maintained the same test accuracy as Float32 model while reducing classifier RAM usage from 1.6 KB to 1.4 KB and Flash usage from 17.7 KB to 15.8 KB. This demonstrates that quantization is an effective optimization technique for TinyML systems.

Overall, the final model achieved greater than 90 % accuracy, fit comfortably with the memory limits of Arduino Nano 33 BLE Sense Rev2, and executed inference with an estimated latency 35 ms. These results demonstrate that this TinyML badminton coach successfully conducted real-time badminton smash classification.

Metric Selection and Justification

Task Type	Metrics Used	Justification
Multi-class classification	Accuracy, weighted F1 score, confusion matrix, RAM usage, Flash/model size, and latency	Accuracy shows overall classification performance, while weighted F1 is useful because it accounts for precision and recall across the three classes. The confusion matrix helps identify which classes are confused with each other. RAM, Flash, and latency are important because the final model must run efficiently on the Arduino Nano 33 BLE Sense Rev2.

Big ML / Baseline Model Performance

Model	Train / Validation Result	Test Result	Key Metrics	Size / Resource Usage	Worked? Why?
Spectral Analysis + Classifier	90.0% validation accuracy	90.0% accuracy / 0.90 F1	Accuracy, F1, confusion matrix	1.4 KB RAM, 15.8 KB Flash, 1 ms classification latency	Yes. Spectral features summarized the IMU motion patterns better than raw time-series inputs and provided better accuracy with lower memory usage.
Raw Data + Classifier	80.0% validation accuracy	80.0% accuracy / 0.80 F1	Accuracy, F1, confusion matrix	2.0 KB RAM, 26.0 KB Flash, 1 ms classification latency	Partially. It worked, but it performed worse than the spectral pipeline and used more memory, so it was not selected for final deployment.

TinyML and Compressed Model Performance

Model	Method	Train	Test	Size / Latency	Notes
Float32 Spectral Model	Unquantized model	N/A	92.31% accuracy	1.6 KB classifier RAM, 17.7 KB Flash, 35 ms total latency	Used as the reference model before quantization.
INT8 Spectral Model	Post-training INT8 quantization	N/A	92.31% accuracy, 0.92 weighted F1	1.4 KB classifier RAM, 15.8 KB Flash, 35 ms total latency	Selected as the final deployed model because it preserved accuracy while reducing classifier RAM and Flash.

Deployment Resource and Performance Summary

Hardware	Model / Flash	RAM / Arena	Latency	Output	Evidence
----------	---------------	-------------	---------	--------	----------

Arduino Nano 33 BLE Sense Rev2	INT8 Spectral Analysis + Classifier, 15.8 KB Flash	3.2 KB total RAM; exported TFLite arena size: 3020 bytes ≈ 3.0 KB	35 ms total latency: 34 ms spectral feature extraction + 1 ms classification	Arduino Serial Monitor class probabilities	Serial Monitor screenshots show live predictions for good_smash , weak_smash , and non_smash .
--------------------------------	--	---	--	--	---

On-Device Inference Evidence

ID	Input / Trial Type	Expected	Predicted	Correct ?	Evidence
1	Good smash swing	good_smash	good_smash	Yes	Serial Monitor screenshot
2	Good smash swing	good_smash	good_smash	Yes	Serial Monitor screenshot
3	Good smash swing	good_smash	good_smash	Yes	Serial Monitor screenshot
4	Weak smash swing	weak_smash	weak_smash	Yes	Serial Monitor screenshot
5	Weak smash swing	weak_smash	weak_smash	Yes	Serial Monitor screenshot
6	Weak smash swing	weak_smash	weak_smash	Yes	Serial Monitor screenshot
7	Non-smash motion	non_smash	non_smash	Yes	Serial Monitor screenshot
8	Non-smash motion	non_smash	non_smash	Yes	Serial Monitor screenshot
9	Non-smash motion	non_smash	non_smash	Yes	Serial Monitor screenshot
10	Additional live test	Demoed all	Predicted all	Yes	Serial Monitor screenshot/log

Challenges, Limitations, and Future Work:

One main limitation was the small dataset size. The final dataset contained 60 total samples, which was enough to demonstrate the TinyML pipeline but not enough to cover many different players, swing styles, racket speeds, or practice conditions. Because of this, the model may perform best on motions similar to the ones collected during the project.

Another limitation was class overlap between *weak_smash* and *non_smash*. These two classes can have similar motion patterns because a weak smash may not produce a strong acceleration or gyroscope signal. This made classification more difficult than detecting a clear *good_smash*. The spectral feature pipeline helped improve separation compared to raw time-series data, but more data would likely be needed to make this distinction more reliable.

The deployment process also required the live sensor input format to match the format used during training. Since the model was named using six IMU channels named *aX*, *aY*, *aZ*, *gX*, *gY*, and *gZ*, the Arduino sensor fusion example had to be adjusted so the live accelerometer and gyroscope readings matched those same input names. This was important because mismatched input channels could prevent the deployed model from running correctly or could lead to incorrect inference results.

Finally, the current system only outputs class probabilities through the Arduino Serial Monitor. This was sufficient for verifying on-device inference, but it is not yet a complete user-facing coaching interface. The system can classify the swing type, but it does not yet explain detailed form issues such as racket angle, posture, timing, or follow-through.

Future work should focus on improving both the dataset quality and the type of feedback the system can provide. First, the dataset could be expanded by collecting more labeled swing samples from experienced or professional badminton players. This would make the model more reliable because the training data would include better examples of proper smash technique, different swing styles, and more variation across players.

Second, the system could be extended beyond IMU-based classification by adding vision-based form analysis. A camera-based model could evaluate racket angle, posture, arm position, and follow-through, while the IMU model continues to classify swing motion. Combining IMU sensing with vision would make the system closer to a real badminton coaching tool because it could provide feedback not only on whether a swing was a good smash, weak smash, or non-smash, but also on why the form may need improvement.

Another future improvement would be adding a more user-friendly feedback output. Instead of only printing probabilities to the Arduino Serial Monitor, the system could send results through BLE, display them on a mobile app, or use LEDs/buzzer feedback during practice. This would make the system easier to use outside of a lab setting.

Writing Quality, Reproducibility, and References:

This report is organized to describe the full TinyML workflow from dataset collection to model training, compression, deployment, and on-device testing. The main implementation was completed using Edge Impulse for feature extraction, model training, testing, INT8 quantization, and Arduino library export. The deployed system used the Arduino Nano 33 BLE Sense Rev2 with onboard accelerometer and gyroscope data as the input source.

To make the project reproducible, the public GitHub repository includes the Arduino deployment sketch, Edge Impulse model/library files, dataset files or dataset link, deployment screenshots, and instructions for running inference on the Nano 33 BLE Sense Rev2. The important configuration choices include a 1000 ms input window, 100 Hz sampling rate, six IMU input channels, three output classes, spectral feature extraction, and INT8 quantized deployment.

References used for this project:

- Edge Impulse Studio – model training, spectral analysis, quantization, and Arduino deployment
- Arduino Nano 33 BLE Sense Rev2 documentation
- Arduino BMI270/BMM150 IMU library documentation
- TensorFlow Lite for Microcontrollers / Edge Impulse EON Compiler
- Team-collected badminton IMU dataset: good_smash, weak_smash, non_smash
- Team-generated Edge Impulse results, deployment screenshots, and Arduino Serial Monitor outputs

Public GitHub Repository:

<https://github.com/btaing14/TinyML-Badminton-Coach/tree/main/TinyML%20Badminton%20Coach>

Public Edge Impulse Link: <https://studio.edgeimpulse.com/studio/1025275>